

Cloud, Edge & Fog Computing

+ Docker, Kubernetes & VM Migration

Complete Beginner-Friendly Revision Notes

Part 1 — Core Concepts

Cloud Computing • Edge Computing • Fog Computing • Cloudlets
How they differ, when to use each, real-life examples

Part 2 — MCQ Deep Dives (Set 9)

10 questions on cloud/fog/edge architecture fully explained
Latency, load balancing, hybrid cloud, security

Part 3 — MCQ Deep Dives (Set 10)

10 questions on VM migration, Docker, Kubernetes fully explained
Pre-copy migration, containers, namespaces, cgroups, Docker Engine

Concept-First • Story-Driven • Exam-Ready

No prior knowledge required

Contents

1	Understanding the Four Computing Paradigms	2
1.1	Cloud Computing — The Faraway Powerhouse	2
1.2	Edge Computing — The Instant Local Helper	3
1.3	Fog Computing — The Smart Middle Layer	3
1.4	Cloudlet — The Nearby Mini-Cloud	4
1.5	Side-by-Side Comparison	5
1.6	The Ultimate Memory Story	5
2	Solution Set 9 — Edge, Fog and Cloud Architecture	6
2.1	Q1 Cloud vs Fog Computing — The Core Difference	6
2.2	Q2 Self-Driving Car — Which Computing Style?	6
2.3	Q3 Order of Computing Layers — Closest to Farthest	7
2.4	Q4 Preventing Overload in Fog Nodes	7
2.5	Q5 Task Distribution — Edge/Fog vs Cloud	8
2.6	Q6 The Key Factor for Choosing Edge Over Cloud	9
2.7	Q7 What Improves When Moving from Cloud to Fog?	9
2.8	Q8 Trust Between Cloud Providers	10
2.9	Q9 How Different Clouds Communicate	10
2.10	Q10 Why Move Workloads in a Hybrid Cloud?	11
3	Solution Set 10 — VM Migration, Docker & Kubernetes	13
3.1	Q1 Live VM Migration — The Biggest Challenge	13
3.2	Q2 Pre-Copy Migration — Why It Uses the Most Bandwidth	14
3.3	Q3 What Increases Energy Consumption in Fog Migration?	14
3.4	Q4 What Does Kubernetes Actually Do?	15
3.5	Q5 Who Reads Dockerfile Instructions?	16
3.6	Q6 How Docker Keeps Containers Isolated	17
3.7	Q7 What Happens to Data When a Container Is Deleted?	18
3.8	Q8 Who Actually Creates and Runs Containers?	18
3.9	Q9 Calculating VM Migration Time	19
3.10	Q10 Do Containers Need a Full Operating System?	20
4	Master Quick-Reference Sheet	22
4.1	Computing Paradigms at a Glance	22
4.2	Key Concepts from Solution Set 9	23
4.3	Key Concepts from Solution Set 10	24
4.4	Five Golden Rules to Never Forget	24

1 Understanding the Four Computing Paradigms

The Big Picture — Before We Start

Think of the internet as a giant helper system. When your phone or laptop needs to do heavy work (like storing videos or recognizing faces), it has to ask for help somewhere. The big question is: **where does that help come from, and how far away is it?**

Depending on how far the processing happens from your device, we get four different computing styles: Cloud, Edge, Fog, and Cloudlet. Each one solves a different problem.

1.1 Cloud Computing — The Faraway Powerhouse

Imagine you need a huge library to research a topic. You don't have that library at home, so you send a request to a massive library in another city. The library people find your answer and send it back. That is cloud computing.

In real terms, **cloud computing** means the heavy work (storing data, running apps, training AI) happens in giant buildings called *data centers*, which are located far away — maybe in another country. You access all of this through the internet.

Real Example — Netflix:

When you press Play on Netflix, the video is not on your phone. Your request travels to a Netflix data center, the server sends the video back to your phone, and you watch it. This entire back-and-forth happens in seconds thanks to fast internet.

Another example: Google Drive, Gmail, and ChatGPT all use cloud computing. Your data is stored on remote servers, and you just access it from anywhere.

The biggest advantage is **power** — cloud servers are incredibly capable. The downside is **distance**. Because the servers are far away, there is always a small delay. In most everyday apps, this delay is invisible. But in real-time situations (like a car braking), even a fraction of a second can be dangerous.

The One Weakness of Cloud:

Distance creates delay (called latency).

Sending data to a server far away and waiting for a reply takes time. For most apps, this is fine. But for anything that needs an instant response, cloud computing alone is not enough.

1.2 Edge Computing — The Instant Local Helper

Now imagine the same situation, but instead of sending homework to a teacher in another city, you solve it yourself right at your desk. That is **edge computing**.

Here, the processing happens **right where the data is created** — on the device itself, or a machine sitting just a few meters away. There is no waiting for data to travel far.

The Self-Driving Car Story:

A self-driving car has cameras and sensors watching the road constantly. Suddenly a child runs in front of the car. Should the car send a message to a faraway server and wait for the server to say "brake"? Absolutely not — by that time, the accident would have already happened.

The car uses edge computing: all the decision-making (detect obstacle → brake immediately) happens inside the car's own computer, in milliseconds. No cloud needed.

More real examples of edge computing:

A smart CCTV camera that detects motion right inside the camera, without sending every frame to the cloud. A factory machine that detects its own temperature and shuts itself down if it overheats. A fitness wristband that tracks your heartbeat in real time.

The Power of Edge Computing:

- ✓ Processing happens on or very near the device
- ✓ Response time is near-zero (no internet delay)
- ✓ Works even without internet connection
- ✓ Reduces the amount of data sent to the cloud

The limitation is that edge devices are **not as powerful as cloud servers**. They can handle quick decisions, but not massive analysis tasks.

1.3 Fog Computing — The Smart Middle Layer

Now think about a huge smart city with thousands of traffic lights, air quality sensors, water pipes, and street cameras — all generating data every second. If everything was sent directly to the cloud, the cloud would be overwhelmed. If everything was processed at each tiny sensor (edge), the sensors would be too weak.

The answer? **Fog computing** — a layer of computing that sits between your devices and the cloud.

Think of it like this:

The **cloud** is a very powerful professor in a university far away.

The **edge** is you, trying to solve everything yourself at your desk.

The **fog** is a knowledgeable teaching assistant available in your classroom — not

as powerful as the professor, but much closer and faster to reach.

Fog nodes are typically things like routers, local gateway servers, or mini servers placed in neighbourhoods. They collect data from devices nearby, filter what is important, process what they can, and only forward the truly necessary data to the cloud.

Smart City Example:

1000 sensors measure pollution levels across the city every second. Instead of sending all 1000 readings every second to the cloud:

- A fog node in each neighbourhood collects readings from its 50 nearby sensors.
- It calculates the average pollution level locally.
- It only sends the summary (1 value instead of 50) to the cloud.

This **reduces cloud traffic by 98%** and speeds up the system dramatically.

What Fog Computing Does:

- ✓ Reduces the load on the cloud
- ✓ Faster than sending everything to the cloud
- ✓ More capable than edge devices for complex filtering
- ✓ Acts as an intelligent pre-processor

1.4 Cloudlet — The Nearby Mini-Cloud

A **cloudlet** is a small, powerful computing server placed very close to users — like at a college campus, a coffee shop, a hospital, or near a mobile tower.

Think of it as a **mini data center** that is locally accessible. It is more powerful than edge devices and fog nodes, but it is physically nearby — not somewhere far across the internet.

AR/VR on Your Phone Example:

You put on AR glasses for a surgery training simulation. This requires heavy 3D processing. Your glasses are not powerful enough (edge limitation), and the cloud is too far (too slow for real-time 3D).

Solution: Your glasses connect to a cloudlet server installed in your hospital building. The cloudlet does the heavy computation and sends the 3D render back to your glasses in milliseconds. **Fast + Powerful + Nearby.**

Cloudlets are especially designed for **mobile users** who move around. As you walk across a campus, your phone seamlessly connects to the nearest cloudlet — similar to how your phone automatically connects to the nearest WiFi router or cell tower.

1.5 Side-by-Side Comparison

Feature	Cloud	Edge	Fog	Cloudlet
Location	Far (data center)	On/near device	Between device & cloud	Nearby building/campus
Speed	Slower (latency)	Fastest (instant)	Fast	Very fast
Power	Highest	Lowest	Medium	Medium-High
Internet needed	Yes	No	Partial	Minimal
Best For	Heavy tasks, storage	Real-time decisions	Filtering, smart cities	Mobile heavy tasks
Example	Netflix, Gmail	Self-driving car	Smart city sensors	AR/VR on campus

1.6 The Ultimate Memory Story

You have a maths problem to solve:

You send it to a professor in another country via post ⇒ **Cloud Computing** (powerful, but slow)

You solve it yourself right now ⇒ **Edge Computing** (instant, but your brain has limits)

You ask a teaching assistant sitting in your classroom ⇒ **Fog Computing** (quick, moderately powerful)

You go to a small study center with powerful computers just outside your building ⇒ **Cloudlet** (fast, powerful, nearby)

None of them is the best for all situations. Each one fits a specific need.

2 Solution Set 9 — Edge, Fog and Cloud Architecture

How to use this section: Each question is explained from scratch — no assumption of prior knowledge. Read the explanation first, then look at the answer. The goal is understanding, not guessing.

2.1 Q1 Cloud vs Fog Computing — The Core Difference

The Question: What is the fundamental difference between cloud computing and fog computing?

Let us think about this carefully. Cloud computing uses very powerful servers located in large data centers, which could be hundreds or thousands of kilometers away. Everything goes there, gets processed, and comes back. This setup is excellent when you have time to wait and need massive computing power.

Fog computing changes the equation. Instead of sending *everything* to faraway servers, fog brings some of the computing power closer to the user — to routers, local servers, or gateway devices in your neighbourhood or building.

Homework Analogy:

Cloud = sending your homework to a brilliant professor in another country by courier. Great results, but you wait days.

Fog = your school has a good teacher right there. Not as expert as the professor, but you get help within the hour.

The key phrase to remember is: fog **brings computation closer to the data source**, while cloud **centralizes computation in remote data centers**.

Correct Answer: Cloud computing uses centralized remote data centers; fog computing brings computation closer to the network edge (the devices generating data).

2.2 Q2 Self-Driving Car — Which Computing Style?

The Question: A self-driving car needs to make split-second braking decisions. Which computing paradigm is most appropriate?

A self-driving car collects data from cameras, radar, and sensors in real time. It processes this data and makes decisions like "slow down", "change lane", or "brake now".

Now ask yourself: can the car send all this data to a cloud server, wait for the server to decide "brake", and then brake? No. By the time the server responded, the car would have already crashed.

Why cloud fails here:

Even with fast internet, sending data to a remote server and receiving a response takes at least 50–200 milliseconds. At highway speed, a car travels 4–6 meters in that time. That delay can be fatal.

The car needs **edge computing**: all the processing happens inside the car's own computer, in real time, without relying on any network connection. The response happens in single-digit milliseconds.

Correct Answer: Edge computing — because safety-critical decisions need instant, local processing with zero network dependency.

2.3 Q3 Order of Computing Layers — Closest to Farthest

The Question: Arrange the computing paradigms in order from closest to farthest from the user's device.

This question tests whether you understand where each layer physically sits.

Visualize it as concentric circles around your device:

Layer 1 — Edge: Right at or on your device (phone camera, car computer, smart watch)

Layer 2 — Fog: Nearby infrastructure (WiFi router, local server, neighbourhood gateway)

Layer 3 — Cloud: Remote data centers (Amazon AWS, Google Cloud, Microsoft Azure)

Think of it like concentric rings: your device is at the center, edge processing is in the innermost ring, fog is the next ring out, and cloud is the outermost — the most powerful but the farthest.

Correct Order: Edge → Fog → Cloud (nearest to farthest from the user device).

2.4 Q4 Preventing Overload in Fog Nodes

The Question: What technique prevents overloading when many IoT devices all send data to fog nodes simultaneously?

Imagine a single fog server that handles a neighbourhood of 500 smart home devices — thermostats, cameras, meters, locks. Now imagine all 500 devices send data at exactly the same moment (say, at midnight when they all sync). The fog server gets flooded.

What is the smart solution? **Spread the work across multiple fog servers** so no single server is overloaded. This is called **load balancing**.

Classroom Analogy:

100 students all rush to ask the same teacher for help at once. The teacher is overwhelmed.

Solution: Open three classrooms with three teachers. Direct students to different rooms based on availability. Each teacher handles a manageable load.

That is load balancing — distributing requests across multiple capable nodes.

Load balancing in fog computing works the same way: a smart controller checks which fog node has more free capacity and routes incoming data to the least-loaded one.

Correct Answer: Load balancing — distributing incoming requests across multiple fog nodes to prevent any single node from being overwhelmed.

2.5 Q5 Task Distribution — Edge/Fog vs Cloud

The Question: How should tasks be divided between edge/fog and cloud?

Not all tasks are created equal. Some tasks are **urgent but light** (quick decisions on small data), and others are **heavy but not urgent** (analyzing months of historical data).

Two categories of tasks:

Time-critical tasks need a response in milliseconds. Examples include: emergency brake signals, real-time video compression, instant fraud alerts, health monitor alarms. These must go to **edge or fog** because there is no time to travel to the cloud.

Delay-tolerant heavy tasks do not need instant results. Examples include: analyzing one year of sales data, training a machine learning model, generating a monthly report. These can go to the **cloud** where there is unlimited power.

Correct Answer: Time-critical tasks go to edge/fog (fast, nearby). Heavy but delay-tolerant tasks go to cloud (powerful, remote). The split is based on **urgency and computational weight**.

2.6 Q6 The Key Factor for Choosing Edge Over Cloud

The Question: What is the most important factor when deciding to use edge computing instead of cloud computing?

When you compare edge and cloud, the cloud usually wins on power — it has more CPU, storage, and software available. But edge wins on one critical dimension: **how fast you need the answer**.

This is called **latency sensitivity**. Latency just means delay — the time between making a request and receiving a response.

Medical Device Example:

A patient has a pacemaker that monitors their heart. If it detects an abnormal rhythm, it must send an electric pulse to correct it within milliseconds.

Sending that signal to a cloud server and waiting for a reply would take too long. The patient could die in that window. So the pacemaker processes data at the edge, right inside the device.

On the other hand, a doctor reviewing a patient's *monthly* heart data report can use a cloud system — there is no urgency.

The deciding factor is always: **“How fast does this need to happen?”** The higher the latency sensitivity, the more you lean toward edge.

Correct Answer: Latency sensitivity — how critical the speed of response is for that particular task.

2.7 Q7 What Improves When Moving from Cloud to Fog?

The Question: What performance metric improves most significantly when shifting from cloud to fog?

The primary thing that changes when you move processing from a faraway cloud to a nearby fog node is **the time it takes for data to travel**. This travel time is called **network latency**.

Cloud servers may be 10,000 km away. Even at the speed of light through fiber optic cables, a round trip takes 60–100 milliseconds. Fog nodes are maybe 1 km away — the round trip is under 1 millisecond.

What fog improves and what it does not:

- ✓ **Latency (response time)** — massively improved, because the data does not travel far
- ✓ **Bandwidth usage** — less data travels to the cloud, freeing up internet bandwidth

- ✗ **Raw processing power** — fog nodes are not as powerful as cloud servers
- ✗ **Long-term storage capacity** — cloud still wins here

Correct Answer: Network latency — data travels a much shorter distance to fog nodes, dramatically reducing response time.

2.8 Q8 Trust Between Cloud Providers

The Question: What is the primary challenge in establishing trust in a multi-cloud environment?

A *multi-cloud environment* means a company uses services from multiple cloud providers — for example, storing data on AWS, running applications on Azure, and doing analytics on Google Cloud.

The challenge: these three providers have **completely different systems, identities, and security rules**. How does each system know and trust that a request from another system is legitimate?

Travel Passport Analogy:

When you travel to three countries, each country needs to verify who you are. They check your passport (identity). Without a recognized passport (standard identity system), you cannot cross the border.

In multi-cloud, if Provider A cannot verify that a request truly comes from an authorized system in Provider B, it cannot trust it. This is why **secure identity management** — a unified way to verify who is making each request — is the core challenge.

The solution involves standardized identity protocols (like OAuth2, OpenID Connect) and strict SLA agreements that define rules for data sharing between providers.

Correct Answer: Establishing secure identity verification and enforcing consistent SLA agreements across different cloud providers with different security frameworks.

2.9 Q9 How Different Clouds Communicate

The Question: What is the key mechanism that allows different cloud systems to interoperate?

Different cloud providers are built by different companies using different technologies. AWS uses its own APIs, Azure uses its own, Google Cloud uses its own. Without a shared language, they cannot talk to each other.

The solution is **standardized interfaces and open protocols** — agreed-upon rules that every system follows, so they can all understand each other regardless of their internal differences.

Language Analogy:

Imagine people from India, France, Japan, and Brazil in the same meeting. Each speaks a different language. They all agree to use English as the common language. Similarly, cloud providers agree to use standard APIs and communication protocols (like REST APIs over HTTPS, JSON data format, OAuth for authentication) so their systems can exchange information without knowing each other's internal details.

Without these standards, you would be completely locked into one provider and could never move data or share resources across clouds — called *vendor lock-in*. Open standards prevent this.

Correct Answer: Standardized interfaces and open protocols (APIs, data formats, authentication standards) that create a common language across different cloud systems.

2.10 Q10 Why Move Workloads in a Hybrid Cloud?

The Question: Why do organizations migrate workloads between private and public clouds in a hybrid setup?

A *hybrid cloud* is when a company uses both a **private cloud** (their own servers, fully controlled) and a **public cloud** (like AWS or Azure, shared with others).

Different workloads have different requirements:

Private Cloud Strengths:

Full security and control. Nobody outside the company can access this. Perfect for sensitive data: medical records, financial data, personal customer information.

Limitation: fixed capacity — you cannot suddenly add 10x servers.

Public Cloud Strengths:

Massive, on-demand scalability. Need 1000 extra servers for one day? Done. Pay only for what you use. **Limitation:** less control over data privacy.

Shopping App During a Festival Sale:

An e-commerce company normally serves 10,000 users per day. During Diwali sale, they get 500,000 users. Their private cloud cannot handle this spike.

Solution: Keep the customer payment data on the private cloud (secure). Move the product browsing and search load to the public cloud (scalable). After the sale, move everything back.

This dynamic shifting is **workload migration** — the core reason for hybrid cloud

architecture.

Correct Answer: To balance security, cost, and scalability — keeping sensitive work in the controlled private cloud and moving burst or non-sensitive workloads to the scalable public cloud as demand fluctuates.

3 Solution Set 10 — VM Migration, Docker & Kubernetes

What this section covers:

How virtual machines are moved from one server to another, and how Docker containers work including isolation, storage, and the role of Kubernetes. Everything explained from the ground up.

3.1 Q1 Live VM Migration — The Biggest Challenge

The Question: What is the primary challenge when migrating a running VM over a Wide Area Network (WAN)?

First, let us understand what *live VM migration* is. A Virtual Machine (VM) is like a computer running inside another computer. Live migration means moving this running VM — all its memory, running programs, and data — from one physical server to another, **without shutting it down** and without the user noticing.

Now imagine doing this when the two servers are in different cities. The memory of a running VM could be 8 GB, 16 GB, or more. This entire memory must be transferred through the network while the VM is still running and users are still using it.

The Core Problem — Network Instability:

Over a WAN (Wide Area Network — internet between cities or countries), two things make this very hard:

High Latency: Every byte of memory takes longer to travel because of the physical distance. What takes 0.1ms on a local network takes 50ms over a WAN.

Bandwidth Variability: Internet speed fluctuates. If the connection suddenly slows, the migration cannot keep up with the VM's memory changes — it falls further and further behind, or fails entirely.

Moving Furniture Analogy:

Imagine trying to move all the furniture in your house to a new house in another city while you are still living in it and using all the furniture. Every time movers pick something up, you might need it again. This is incredibly hard — and that is what live VM migration faces over a WAN.

Correct Answer: High network latency and bandwidth variability — the main challenge is transferring large amounts of live memory reliably across an unstable, long-distance network connection.

3.2 Q2 Pre-Copy Migration — Why It Uses the Most Bandwidth

The Question: Which migration technique causes the highest network bandwidth consumption?

There are several techniques to migrate a VM. The most common is called **pre-copy migration**. Understanding why it uses so much bandwidth requires understanding how it works.

In pre-copy migration, the process works in rounds:

Round-by-Round Explanation:

Round 1: Copy all the VM's memory to the destination server. This might be 16 GB.

Problem: While copying, the VM is still running. Some pages of memory that were already copied have now been **changed** (called “dirty pages”).

Round 2: Copy only the dirty pages again — the ones that changed during Round 1.

Problem: While copying those dirty pages, *more* pages get dirty again.

Round 3, 4, 5...: Keep copying dirty pages from the previous round.

This continues until the number of dirty pages is small enough that the VM can be paused for a final tiny transfer.

The issue is clear: pages get copied, re-copied, and re-copied again. A busy VM (running a database, for example) generates dirty pages faster than they can be copied. This **repeated re-transmission** is what makes pre-copy the biggest bandwidth consumer among all migration techniques.

Correct Answer: Pre-copy migration — because it iteratively re-copies memory pages that change during each round of copying, leading to significantly more data being transmitted than the actual VM memory size.

3.3 Q3 What Increases Energy Consumption in Fog Migration?

The Question: What is the primary cause of increased energy consumption during VM migration in fog environments?

Fog environments have a unique challenge: fog nodes have limited power — both in terms of computational capacity and, in some cases, literal electrical power (they might be battery-powered edge nodes or IoT gateways).

When pre-copy migration is happening on a fog node, the problem of dirty pages (discussed above) is magnified. Every time a memory page changes, it must be re-transmitted. In a busy system, this means the network interface is constantly working, the CPU is constantly copying memory, and all of this burns **energy continuously**.

Why repeated dirty page transmission wastes energy:

Each network transmission requires the CPU to gather the data, the network card to send it, and the receiving end to process it. When the same data is sent 3, 4, or 5 times because it keeps changing, you are doing 3–5 times the work for no extra benefit. All that extra work = extra energy burned.

On a fog node with limited battery or power budget, this is a serious problem.

Correct Answer: Frequent retransmission of dirty memory pages — constantly copying and re-copying memory that keeps changing causes the CPU and network hardware to work much more than necessary, wasting energy.

3.4 Q4 What Does Kubernetes Actually Do?

The Question: Which of the following best describes Kubernetes' function?

Before understanding Kubernetes, you need to understand [containers](#). A container is like a small, self-contained box that holds your application and everything it needs to run (code, libraries, settings). Docker creates these containers.

Now imagine you have **hundreds of these containers** running across **dozens of servers**. Questions immediately arise: Which container runs on which server? What if a server crashes? What if traffic spikes and you need 50 more containers? Who monitors health?

Kubernetes is the answer to all these questions. It is an [orchestration system](#) — a smart manager for containers.

What Kubernetes Does — Explained Simply:

Scheduling: Decides which physical server each container should run on based on available resources.

Scaling: If your app suddenly gets 10x traffic, Kubernetes automatically starts more container instances. When traffic drops, it removes extra ones.

Self-Healing: If a container crashes, Kubernetes notices immediately and restarts it. If a whole server dies, Kubernetes moves the containers to other healthy servers.

Load Balancing: Distributes incoming network traffic evenly across all container instances.

Rolling Updates: Updates your app gradually — replacing old containers with new ones one by one, so users never experience downtime.

Food Delivery Platform Analogy:

Imagine 100 kitchens (servers) making food for 1000 orders (containers). Kubernetes is the central operations manager who: assigns orders to the right kitchens, opens more kitchens when demand spikes, immediately replaces a kitchen that breaks down, and ensures no kitchen is overloaded while others are idle. Without this manager, the operation collapses.

Correct Answer: Kubernetes orchestrates, schedules, scales, and manages containers across multiple servers — providing automated deployment, self-healing, and load balancing.

3.5 Q5 Who Reads Dockerfile Instructions?

The Question: Which component reads the Dockerfile and builds an image?

Let us understand the Docker workflow from scratch.

A *Dockerfile* is a text file containing step-by-step instructions for building an application image. Think of it as a recipe: “Start with Ubuntu Linux. Install Python. Copy my app files. Run the app.”

A *Docker Image* is the result of following those instructions — a ready-to-run package.

A *Docker Container* is a running instance of that image.

Now there are three actors in Docker:

Component	Role
Docker CLI	The command-line tool you type commands into (e.g., <code>docker build</code> , <code>docker run</code>)
Docker Daemon	The background service that actually does the work
Docker Hub	The online registry where images are stored and shared

When you type `docker build`, the Docker CLI sends this request to the **Docker Daemon**. The Daemon is the one that actually opens the Dockerfile, reads each instruction, and executes them one by one to create the image.

Restaurant Kitchen Analogy:

You (Docker CLI) write a recipe card (Dockerfile) and hand it to the kitchen ordering system.

The head chef (Docker Daemon) receives the order, reads the recipe step by step, and actually prepares the dish (builds the image).

The menu board (Docker Hub) is where completed dishes are listed for others to order.

Correct Answer: The Docker Daemon — it receives the build request from the CLI and processes the Dockerfile line-by-line to create the container image.

3.6 Q6 How Docker Keeps Containers Isolated

The Question: What mechanism does Docker use to isolate containers from each other on the same host?

Multiple containers run on the same physical machine (host). Yet they do not interfere with each other — one container cannot read another container's files, cannot see another's network traffic, and cannot steal another's CPU or memory. How?

Docker uses two fundamental Linux technologies:

1. Linux Namespaces — “Private Rooms”:

A namespace creates a private, isolated view of a system resource for each container. For example: The **PID namespace** makes each container think it is the only process running — it cannot see processes from other containers. The **Network namespace** gives each container its own private network interface, IP address, and port space. The **Filesystem namespace** (mount namespace) gives each container its own isolated view of the file system.

2. cgroups (Control Groups) — “Resource Budgets”:

cgroups limit how much CPU, memory, and network bandwidth each container can use. If Container A is allowed 1 GB of RAM, it cannot use more even if the host has 64 GB available. This prevents one noisy container from starving others of resources.

Apartment Building Analogy:

All residents live in the same building (same physical server). Each apartment (namespace) has its own lock, its own address, its own rooms — residents cannot walk into each other's apartments.

The building manager (cgroups) sets limits: each apartment gets exactly 200 liters of water per month and cannot use more, no matter how much water is in the building's tank.

Same building, completely separate and fair living — that is how namespaces and cgroups work together.

Correct Answer: Linux namespaces (for isolation of processes, network, and filesystem) and cgroups (for limiting resource usage like CPU and memory) together provide container isolation.

3.7 Q7 What Happens to Data When a Container Is Deleted?

The Question: Assess the following statements about Docker data persistence.

This is a very important concept for practical use. Docker has two places where data can live:

Place 1 — Inside the Container (Ephemeral Storage):

When a container runs, it has its own writable layer on top of the image. You can write files here, create databases, etc. However, this layer is **tied to the container's lifetime** . The moment you delete the container with `docker rm`, everything inside this writable layer is **permanently gone** .

Place 2 — Docker Volumes (Persistent Storage):

A Docker volume is a dedicated storage area managed by Docker, stored outside the container's own filesystem. You attach a volume to a container: `docker run -v myvolume:/data`.

Data written to `/data` goes into the volume. Even if you delete the container, the volume survives and can be attached to a new container later. This is the correct way to store important data in Docker.

Whiteboard vs External Hard Drive:

Writing data inside a container is like writing on a whiteboard that is bolted to a specific room. When the room (container) is demolished, the whiteboard and everything on it is gone.

Using a Docker volume is like saving your work to an external hard drive. You can take that drive out, give it to a different computer (new container), and all your data is still there.

Correct Answer: Statement 1 is true (data in Docker volumes persists after container deletion); Statement 2 is false (data stored only inside the container is permanently lost when the container is deleted).

3.8 Q8 Who Actually Creates and Runs Containers?

The Question: Which component is responsible for creating, running, and managing containers at runtime?

We met the Docker Daemon in Question 5. Now let us go one level deeper.

Docker Engine is the complete system that makes Docker work. It consists of several parts, but the central actor that physically creates containers, pulls images, manages volumes, and handles networking is the **Docker Engine** (specifically its daemon component — sometimes referred to together as Docker Engine).

Component	What it does
Docker CLI	Interface for users to type commands — passes requests to the Daemon
Docker Daemon	Runs as a background service; receives CLI requests and executes them
containerd	Lower-level runtime that actually starts/stops containers
Docker Hub	Online image registry (like an app store for images)
Docker Engine	The complete package: CLI + Daemon + containerd working together

Car Engine Analogy:

You press the accelerator (CLI) → the engine receives the signal → combustion happens, wheels spin (Daemon + containerd create and run containers).

You can see the dashboard and press pedals (CLI), but the actual physical work is done by the engine (Docker Engine). Without it, no car moves.

Correct Answer: Docker Engine (specifically the Docker Daemon at its core) creates, manages, and runs containers at runtime in response to commands from the CLI.

3.9 Q9 Calculating VM Migration Time

The Question: A VM has 64 GB of memory. The network link speed is 8 MB/s. Calculate total migration time and total downtime for a non-live migration.

First, let us understand **non-live (cold) migration**. In this type, the VM is **completely shut down** first. Then all its memory/disk is copied to the new server. Then the VM is started again. The VM is offline for the entire duration of the copy — this is the **downtime**.

This is simpler than live migration because nothing is changing during the copy — no dirty pages, no re-copying.

Step-by-step calculation:

$$\text{Total data to transfer} = 64 \text{ GB} = 64 \times 1024 \text{ MB} = 65,536 \text{ MB}$$

$$\text{Network speed} = 8 \text{ MB/s}$$

$$\text{Transfer time} = \frac{\text{Total data}}{\text{Speed}} = \frac{65,536 \text{ MB}}{8 \text{ MB/s}} = 8,192 \text{ seconds}$$

$$8,192 \text{ seconds} \div 3600 \approx \boxed{2.27 \text{ hours} \approx 2 \text{ hours}}$$

Key Insight about Downtime:

In non-live (cold) migration, the VM is OFF for the entire transfer. So:

Total migration time = 2 hours

Total downtime = 2 hours (same, because VM is off throughout)

This is the main disadvantage of cold migration — long downtime. Live migration solves this but at the cost of more network usage.

Correct Answer: Total migration time $\approx$ 2 hours; Total downtime $\approx$ 2 hours (equal, because VM is powered off for the entire transfer in a non-live migration).

3.10 Q10 Do Containers Need a Full Operating System?

The Question: True or False — Each container requires its own full operating system.

Answer: FALSE.

This is one of the most fundamental concepts in container technology, and understanding it explains why containers are so much lighter and faster than virtual machines.

Virtual Machines vs Containers:**Virtual Machines (VMs):**

Each VM includes a complete, full operating system (OS kernel + system libraries + all OS files). This makes each VM several GBs in size and slow to start (minutes). The OS is duplicated for every VM on the host.

Containers:

All containers on the same host **share the host's operating system kernel**. The container only contains the application code and its specific dependencies — not a full OS. This makes containers tiny (MBs instead of GBs) and fast to start (seconds or less).

House vs Room Analogy:

A Virtual Machine is like building a complete separate house for each tenant — full foundation, walls, roof, plumbing, electricity. Expensive and slow to build.

A Container is like dividing one house into rooms. Each tenant gets their own space, but they share the house's foundation and plumbing (the OS kernel). Much faster and cheaper to set up.

Important Distinction:

✗ Containers do NOT have their own OS kernel

✓ Containers DO have their own filesystem, processes, and network (via namespaces)

The kernel is **shared from the host** — this is what makes containers lightweight. But it also means all containers must be compatible with the host OS kernel (e.g., Linux containers must run on a Linux host).

Correct Answer: FALSE. Containers share the host operating system's kernel. They only package application code and dependencies — not a full OS. This is what makes them fast, lightweight, and portable.

4 Master Quick-Reference Sheet

4.1 Computing Paradigms at a Glance

Paradigm	Core Idea	Best Use Case
Cloud	Powerful faraway servers	Storage, AI training, big analysis
Edge	Processing on/near the device	Self-driving cars, health monitors
Fog	Middle layer between device and cloud	Smart cities, IoT filtering
Cloudlet	Nearby mini-cloud server	AR/VR, mobile heavy computation

4.2 Key Concepts from Solution Set 9

Topic	Core Concept	Exam Answer
Cloud vs Fog	Location of processing	Cloud = remote; Fog = nearby
Self-driving car	Instant decisions needed	Edge computing
Layer order	Nearest to farthest	Edge → Fog → Cloud
Load balancing	Prevents fog overload	Distribute across multiple fog nodes
Task division	Time-critical vs heavy	Edge/fog vs cloud
Choosing edge	Deciding factor	Latency sensitivity
Fog benefit	What it improves	Network latency (response speed)
Multi-cloud trust	Main challenge	Secure identity + SLA enforcement
Interoperability	How clouds communicate	Standard APIs and open protocols
Hybrid cloud	Reason for workload move	Balance security, cost, scalability

4.3 Key Concepts from Solution Set 10

Topic	Core Concept	Exam Answer
Live VM migration	WAN challenge	High latency and bandwidth variability
Pre-copy migration	Why it uses most bandwidth	Repeated dirty page retransmission
Fog energy waste	Root cause	Frequent dirty page retransmission
Kubernetes	What it does	Orchestrate, schedule, scale, manage containers
Dockerfile builder	Which component	Docker Daemon
Container isolation	Mechanism	Linux namespaces + cgroups
Data persistence	Volume vs internal	Only volumes survive container deletion
Container creator	Runtime component	Docker Engine (Daemon)
Migration time calc	64GB at 8MB/s (cold)	≈ 2 hours; downtime = 2 hours
Containers need OS?	Full OS required?	FALSE — share host OS kernel

4.4 Five Golden Rules to Never Forget

Rule 1: The closer the processing to the device, the faster the response. Edge is fastest, cloud is slowest for latency.

Rule 2: Fog does not replace cloud or edge — it complements both. It reduces cloud traffic and is more capable than tiny edge devices.

Rule 3: In VM migration, pre-copy creates the most network traffic because dirty pages keep getting re-sent. Non-live migration = downtime equals transfer time.

Rule 4: Containers are lightweight because they share the host OS kernel. VMs are heavy because each one has its own full OS.

Rule 5: Docker Daemon does the actual work. Kubernetes manages many containers. Docker volumes save data permanently. Data inside containers is temporary.